খুব ভালো প্রশ্ন 👍
Django / DRF-এ URL থেকে ViewSet call করার সময় `as_view()` কখন লাগবে, কখন লাগবে না — এটা অনেকেরই confusion থাকে। আমি step-by-step পরিষ্কার করি।

1️⃣ `as_view()` কখন use করবো?

✅ Class-based View (APIView, GenericAPIView, normal View)

এগুলো **directly class**, তাই Django বুঝে না কোন method call করবে।
তাই `as_view()` **বাধ্যতামূলক**।

উদাহরণ (APIView)

```
# views.py
from rest_framework.views import APIView
from rest_framework.response import Response

class MyAPIView(APIView):
    def get(self, request):
        return Response({"msg": "Hello"})
```

```
# urls.py
from django.urls import path
from .views import MyAPIView

urlpatterns = [
    path('api/', MyAPIView.as_view()),  # ✅ as_view() লাগবে
]
```

📌 **Rule:**
👉 **Class-based view হলে → `as_view()` লাগবেই**

2️⃣ ViewSet হলে কখন `as_view()` use করবো?

🔴 Normally: Manual call করলে

ViewSet নিজে থেকে HTTP method handle করে না,
তুমি যদি নিজে URL bind করো → তখন `as_view({})` **দিতে হবে**।

উদাহরণ (Manual ViewSet binding)

```python
# views.py
from rest_framework.viewsets import ViewSet
from rest_framework.response import Response

class BookViewSet(ViewSet):
    def list(self, request):
        return Response({"books": []})

    def retrieve(self, request, pk=None):
        return Response({"book": pk})
```

```python
# urls.py
from django.urls import path
from .views import BookViewSet

urlpatterns = [
    path(
        'books/',
        BookViewSet.as_view({'get': 'list'})  # ✅ as_view দরকার
    ),
    path(
        'books/<int:pk>/',
        BookViewSet.as_view({'get': 'retrieve'})
    ),
]
```

📌 এখানে:

- `'get'` → HTTP method
- `'list' / 'retrieve'` → ViewSet method

👉 **Manual URL + ViewSet = `as_view()` লাগবে**

3️⃣ ViewSet হলে কখন `as_view()` লাগবে না?

🟢 Router ব্যবহার করলে

DRF Router সব কিছু **auto generate করে দেয়**
(HTTP method + URL mapping)

উদাহরণ (Router)

```python
# urls.py
from rest_framework.routers import DefaultRouter
from .views import BookViewSet
```

```
router = DefaultRouter()
router.register('books', BookViewSet, basename='book')

urlpatterns = router.urls
```

এখানে তুমি:

- ❌ `as_view()` লেখো না
- ❌ HTTP method mapping লেখো না

📌 Router internally করে:

```
GET    /books/        → list()
GET    /books/1/      → retrieve()
POST   /books/        → create()
PUT    /books/1/      → update()
DELETE /books/1/      → destroy()
```

👉 **Router + ViewSet = `as_view()` লাগবে না**

4️⃣ One-line Golden Rule 🧠

```
APIView / GenericAPIView → as_view() লাগবে
ViewSet + Manual URL     → as_view({}) লাগবে
ViewSet + Router         → as_view() লাগবে না
```

5️⃣ Interview ready summary 🎯

> **Q:** Why ViewSet doesn't need `as_view()` with router?
> **A:** Because router automatically maps HTTP methods to ViewSet actions and internally calls `as_view()`.

চাও তো আমি

- ✔️ diagram দিয়ে explain
- ✔️ ModelViewSet vs ViewSet difference
- ✔️ Real project-এ কোনটা কখন use করবো

বললেই হবে 😊

না, `@api_view` **use করলে** `as_view()` **লাগবে না।**
এটা DRF-এর **function-based view decorator**, তাই class না।

1️⃣ Function-based APIView

```
# views.py
from rest_framework.decorators import api_view
from rest_framework.response import Response

@api_view(['GET'])
def hello(request):
    return Response({"msg": "Hello"})
```

```
# urls.py
from django.urls import path
from .views import hello

urlpatterns = [
    path('hello/', hello),  # ✅ as_view() লাগবে না
]
```

📌 **Rule:**

- **Class-based view →** `as_view()` **লাগবে**
- **Function-based view + @api_view →** `as_view()` **লাগবে না**

2️⃣ কনফিউশন যেটা হয়

Class-based view:

```
path('hello/', HelloAPIView.as_view())  # class → as_view() লাগবে
```

Function-based view:

```
path('hello/', hello)  # function → as_view() লাগে না
```

💡 **Tip:**

- যদি তুমি **ছোটো API** বানাও → `@api_view` বেশ সহজ

- যদি **CRUD + reuse + generic behavior** দরকার → **ViewSet / GenericAPIView** use করো

যদি চাও, আমি diagram দিয়ে দেখাতে পারি `@api_view` **vs ViewSet vs GenericAPIView** `URL call` **flow**, তখন একটিতে সব confusion পরিষ্কার হবে।
চাও কি আমি সেটা বানাই?

খুব ভালো 👍
APIView দিয়ে media file (image / video / pdf / any file) upload করা ১০০% possible।
এখানে আমি **production-style, complete example** দেখাচ্ছি — TODO app এর সাথে file upload যুক্ত করে।

📦 APIView দিয়ে Media File Upload (Complete Guide)

আমরা কী করবো?

- Image / File upload
- Auth required
- User-wise ownership
- APIView + Serializer

1️⃣ settings.py (MEDIA setup) — MUST

```
import os

MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

Root urls.py

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    ...
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

📌 Development only (production এ S3 / Cloudinary)

2️⃣ Model (models.py)

```
from django.db import models
from django.contrib.auth.models import User

class Todo(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    title = models.CharField(max_length=200)
    attachment = models.FileField(
        upload_to='todos/',
        null=True,
        blank=True
    )
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

- Image হলে `ImageField`
- Video / PDF হলে `FileField`

> ImageField ব্যবহার করলে `pip install pillow`

3️⃣ Serializer (serializers.py)

```
from rest_framework import serializers
from .models import Todo

class TodoSerializer(serializers.ModelSerializer):

    class Meta:
        model = Todo
        fields = ['id', 'title', 'attachment', 'created_at']
```

4️⃣ APIView (views.py)

```
from rest_framework.views import APIView
from rest_framework.parsers import MultiPartParser, FormParser
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework import status
```

```
from .models import Todo
from .serializers import TodoSerializer

class TodoUploadAPIView(APIView):
    permission_classes = [IsAuthenticated]
    parser_classes = [MultiPartParser, FormParser]

    def post(self, request):
        serializer = TodoSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save(user=request.user)
            return Response(serializer.data,
status=status.HTTP_201_CREATED)
        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)
```

🔑 **parser_classes খুব important**
না দিলে file যাবে না

5️⃣ URL (urls.py)

```
from django.urls import path
from .views import TodoUploadAPIView

urlpatterns = [
    path('todos/upload/', TodoUploadAPIView.as_view()),
]
```

6️⃣ Postman / Frontend Request

Headers

```
Authorization: Token xxxxx
```

Body → form-data

Key	Type	Value
title	Text	Learn File Upload
attachment	File	demo.pdf / image.png

✔ JSON নয়
✔ **form-data**

7️⃣ Response Example

```
{
  "id": 1,
  "title": "Learn File Upload",
  "attachment": "/media/todos/demo.pdf",
  "created_at": "2026-01-22T10:30:00Z"
}
```

8️⃣ Image only restriction (Optional but common)

```
class ImageUploadSerializer(serializers.ModelSerializer):

    def validate_attachment(self, file):
        if not file.content_type.startswith('image'):
            raise serializers.ValidationError("Only image allowed")
        return file
```

9️⃣ Multiple files upload (Advanced)

```
def post(self, request):
    files = request.FILES.getlist('attachment')
    for f in files:
        Todo.objects.create(
            user=request.user,
            title=request.data.get('title'),
            attachment=f
        )
```

🧠 Interview One-liner

> **File upload in DRF requires MultiPartParser and form-data request**

❗ Common Mistakes

❌ JSON body দিয়ে file পাঠানো
❌ `parser_classes` না দেওয়া
❌ MEDIA settings না করা
❌ ImageField + pillow install না করা

🚀 Production Tips

- Large file → chunk upload
- Use S3 / Cloudinary
- Validate file size
- Virus scan (enterprise)

🎯 Conclusion

👉 **APIView দিয়েই image, video, pdf, any media fully handle করা যায়**
👉 তুমি এখন **real backend developer level** এ চলে এসেছো 🔥

তুমি চাইলে next step এ আমি দেখাতে পারি 👇

- 🔐 File upload + permission (only owner download)
- 📥 File download API
- ☁️ Cloudinary / S3 integration
- 🔄 Same thing using ViewSet

বললেই হবে 😊

ঠিক আছে 👍
এখন **APIView দিয়ে `.mp4` video upload**–এর **complete + clean example** দিচ্ছি।
এটা image/file upload-এর মতোই, শুধু **video-specific validation** add করবো।

🎥 APIView দিয়ে `.mp4` Video Upload (Complete)

✔ What this supports

- Only `.mp4` video
- Auth required
- User-wise ownership
- APIView + Serializer

1️⃣ settings.py (MEDIA config)

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

```
# root urls.py
from django.conf import settings
from django.conf.urls.static import static

urlpatterns += static(
    settings.MEDIA_URL,
    document_root=settings.MEDIA_ROOT
)
```

2️⃣ Model (models.py)

```
from django.db import models
from django.contrib.auth.models import User

class Video(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    title = models.CharField(max_length=200)
    video = models.FileField(upload_to='videos/')
    uploaded_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

3️⃣ Serializer (serializers.py)

```
from rest_framework import serializers
from .models import Video

class VideoSerializer(serializers.ModelSerializer):
```

```
    def validate_video(self, file):
        if file.content_type != 'video/mp4':
            raise serializers.ValidationError("Only MP4 videos are
allowed")
        if file.size > 50 * 1024 * 1024:  # 50MB
            raise serializers.ValidationError("Max file size is 50MB")
        return file

    class Meta:
        model = Video
        fields = ['id', 'title', 'video', 'uploaded_at']
```

4️⃣ APIView (views.py)

```
from rest_framework.views import APIView
from rest_framework.parsers import MultiPartParser, FormParser
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework import status

from .serializers import VideoSerializer

class VideoUploadAPIView(APIView):
    permission_classes = [IsAuthenticated]
    parser_classes = [MultiPartParser, FormParser]

    def post(self, request):
        serializer = VideoSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save(user=request.user)
            return Response(serializer.data,
status=status.HTTP_201_CREATED)
        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)
```

🔑 `.mp4` upload-এর জন্য
👉 `MultiPartParser` **must**

5️⃣ URL (urls.py)

```
from django.urls import path
from .views import VideoUploadAPIView
```

```
urlpatterns = [
    path('videos/upload/', VideoUploadAPIView.as_view()),
]
```

6️⃣ Postman / Frontend Request

Headers

```
Authorization: Token xxxxxx
```

Body → form-data

Key	Type
title	Text
video	File (.mp4)

❌ JSON body নয়
✅ form-data only

7️⃣ Response Example

```
{
  "id": 1,
  "title": "Django DRF Tutorial",
  "video": "/media/videos/tutorial.mp4",
  "uploaded_at": "2026-01-22T11:20:00Z"
}
```

🔐 Security + Best Practice

✔ MIME + extension দুটোই check

```
if not file.name.endswith('.mp4'):
    raise serializers.ValidationError("Invalid file extension")
```

✔ Large video handling

- Limit size
- Async processing (Celery)
- Stream serving (NGINX)

🎯 Interview One-liner

> **Video upload in DRF uses FileField with MultiPartParser and form-data requests**

🚀 Production Upgrade (Next steps)

- 🎞 Video streaming API
- ☁️ AWS S3 / Cloudinary video
- ⏳ Chunk upload
- 🔄 ViewSet version

তুমি বলো—**upload only**, না **play/stream API**-ও চাই?